

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K. SUHAS VARMA

Roll No.: A24126552091

Date: 29-08-2026

1. TITLE

Develop a Dynamic Webpage Using DOM & JavaScript

2. AIM

To develop a dynamic **Student Form dashboard** webpage using **HTML, CSS, JavaScript, and DOM manipulation** that provides a Task Manager (using OOP and LocalStorage), a User Registration form with dynamic submission display, and a Sports & Interests selector.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of the Document Object Model (DOM).
- To access HTML elements using JavaScript.
- To handle user interactions using event listeners.
- To dynamically create, display, and remove HTML elements.
- To understand Object-Oriented Programming in JavaScript using classes and objects.
- To store and retrieve data using the browser LocalStorage.
- To handle form submissions and display the submitted details dynamically.
- To read values from select boxes and checkboxes and display them as tags.
- To perform input validation and prevent XSS by escaping user input.
- To use JavaScript arrays and objects for storing data.
- To understand how JavaScript can change webpage content without reloading the page.

4. THEORY

The **Document Object Model (DOM)** is a programming interface that represents an HTML document as a tree of objects. JavaScript can use the DOM to access, modify, add, or remove elements and content dynamically.

In this experiment, a **Student Form dashboard** is developed using HTML, CSS, and JavaScript. HTML provides the structure of the three cards, CSS provides the dark-theme appearance using CSS variables, Grid, and Flexbox, and JavaScript handles the dynamic behaviour. The dashboard contains three modules:

- **Task Manager:** A **TaskManager** class (Object-Oriented Programming) stores task objects (*id* and *title*) in an array. The tasks are saved in the browser **LocalStorage** using `JSON.stringify()` and loaded again using `JSON.parse()`, so they remain after the page is reloaded.
- **User Registration Form:** When the form is submitted, the name, email, and role are read, stored in an object with the submission time, and displayed as a card without reloading the page.
- **Sports & Interests Selector:** The selected category and the checked sports are read from the select box and checkboxes and displayed as removable tags (pills).

JavaScript accesses HTML elements using `document.getElementById()` and `document.querySelectorAll()`. The application uses the following DOM methods and properties:

- `getElementById()` / `querySelectorAll()` – select HTML elements.
- `createElement()` – creates a new HTML element dynamically.
- `appendChild()` / `prepend()` – adds a new element at the end or the beginning of a parent element.
- `innerHTML` – modifies the HTML content of an element.
- `dataset` – stores a custom data value (task id) on an element.
- `classList.contains()` – checks whether an element has a given CSS class.
- `remove()` – removes an element from the page.

Event listeners are used to respond to user actions. The *submit* event is used for the three forms, and `e.preventDefault()` stops the page from reloading. **Event delegation** is used for the Delete buttons and tag remove buttons: one listener is added to the parent element and `e.target` identifies the clicked button.

Input validation and security: The `required` attribute, `trim()`, and an alert for an empty sport selection validate the input. The `escapeHTML()` function converts special characters to HTML entities so that user input cannot inject HTML or scripts (XSS protection).

The basic working flow is:

User Input → JavaScript Event → DOM Manipulation → Update Display (and LocalStorage) → Display Updated Result

5. ALGORITHM / PROCEDURE

1. Create a project folder for the Student Form dashboard application.
2. Create an HTML file named `index.html` (CSS and JavaScript are written inside it).
3. Design the page header with the title and subtitle using HTML elements.
4. Create a grid layout containing three cards (sections) using CSS Grid.
5. Card 1: create the Task Manager form with a text input, an Add Task button, and an unordered list.
6. Card 2: create the User Registration form with Full Name, Email, Role (select), a Submit button, and a container for submissions.
7. Card 3: create the Sports & Interests form with a category select, six sport checkboxes, a Save button, and a container for tags.
8. Style the page using CSS variables, dark theme colours, cards, buttons, list items, submission cards, and tag pills.
9. Write the `escapeHTML()` utility function to protect against XSS.

10. Create the TaskManager class with constructor(), addTask(), deleteTask(), and save() methods.
11. Load the saved tasks from LocalStorage and create the appTasks object.
12. Access the required HTML elements using document.getElementById().
13. Write renderTasks() and appendTaskDOM() to display tasks using createElement() and appendChild().
14. Add a submit event listener to the task form to validate input, add the task, and clear the input.
15. Add a click event listener to the task list to delete a task using event delegation.
16. Add a submit event listener to the registration form to read the name, email, and role, and create a submission object.
17. Write renderSubmissionDetails() to display the submission card using prepend(), then reset the form.
18. Add a submit event listener to the interests form to read the category and the checked sports using querySelectorAll().
19. Show an alert if no sport is selected; otherwise call renderInterestTags().
20. Write renderInterestTags() to clear old tags, display the category, and create a pill for each sport.
21. Add a click event listener to the tags container to remove a tag when its × button is clicked.
22. Call renderTasks() on page load, then execute the webpage in a browser and verify all dynamic operations.

6. PROGRAM

index.html (HTML, CSS, and JavaScript in a single file)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>JS Core Concepts Dashboard</title>
  <style>
    :root {
      --bg-color: #0f172a;
      --card-bg: #1e293b;
      --text-color: #f8fafc;
      --accent: #38bdf8;
      --accent-hover: #0284c7;
      --border: #334155;
    }
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-color);
      padding: 20px;
      line-height: 1.6;
    }
    header {
      text-align: center;
      margin-bottom: 30px;
    }
    header h1 {
      color: var(--accent);
      font-size: 2.2rem;
    }
    .grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
      gap: 20px;
      max-width: 1200px;
      margin: 0 auto;
    }
    .card {
      background: var(--card-bg);
      border: 1px solid var(--border);
      border-radius: 12px;
      padding: 20px;
      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.3);
    }
    .card h2 {
      font-size: 1.25rem;
      margin-bottom: 15px;
      color: var(--accent);
      border-bottom: 1px solid var(--border);
      padding-bottom: 8px;
    }
    input[type="text"],
    input[type="email"],
    select,
    button {
      width: 100%;
      padding: 10px;
      margin-top: 8px;
      border-radius: 6px;
      border: 1px solid var(--border);
      outline: none;
    }
    input[type="text"],
    input[type="email"],
    select {
      background: #0f172a;
      color: var(--text-color);
    }
    button {
      background: var(--accent);
      color: #000;
      font-weight: bold;
      cursor: pointer;
      transition: background 0.2s;
    }
  </style>
</head>
<body>
  <div>
    <div>
      <h1>JS Core Concepts Dashboard</h1>
    </div>
    <div>
      <div>
        <h2>Form Elements</h2>
        <input type="text" value="Name" />
        <input type="email" value="Email" />
        <select>
          <option>Select an option</option>
        </select>
        <button type="button">Submit</button>
      </div>
    </div>
  </div>
</body>
</html>
```

```

button:hover {
  background: var(--accent-hover);
  color: #fff;
}
/* Task List */
ul {
  list-style: none;
  margin-top: 15px;
  max-height: 180px;
  overflow-y: auto;
}
li {
  background: #0f172a;
  padding: 8px 12px;
  margin-bottom: 6px;
  border-radius: 4px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.delete-btn {
  background: #ef4444;
  color: white;
  border: none;
  width: auto;
  padding: 4px 8px;
  margin-top: 0;
  font-size: 0.8rem;
}
.delete-btn:hover {
  background: #dc2626;
}
/* Form Submissions Output */
.submission-card {
  background: #0f172a;
  border-left: 3px solid var(--accent);
  padding: 10px 12px;
  margin-top: 10px;
  border-radius: 4px;
  font-size: 0.9rem;
}
.submission-card strong {
  color: var(--accent);
}
.submission-card .timestamp {
  font-size: 0.75rem;
  color: #94a3b8;
  display: block;
  margin-top: 4px;
}
/* Sports & Interests Component */
.checkbox-group {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 8px;
  margin-top: 10px;
  background: #0f172a;
  padding: 10px;
  border-radius: 6px;
  border: 1px solid var(--border);
}
.checkbox-label {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 0.85rem;
  cursor: pointer;
}
.checkbox-label input {
  accent-color: var(--accent);
  cursor: pointer;
}
.tags-container {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
  margin-top: 12px;
}
.tag-pill {
  background: #0f172a;
  border: 1px solid var(--accent);
  color: var(--accent);
  padding: 4px 10px;
  border-radius: 20px;
  font-size: 0.8rem;
}

```

```

        display: flex;
        align-items: center;
        gap: 6px;
    }
    .tag-pill button {
        background: none;
        border: none;
        color: #ef4444;
        width: auto;
        padding: 0;
        margin: 0;
        cursor: pointer;
        font-size: 0.9rem;
        line-height: 1;
    }
}
</style>
</head>
<body>
    <header>
        <h1>Student Form</h1>
        <p>OOP, Form Submissions, DOM State & Interests Selector</p>
    </header>
    <main class="grid">
        <!-- 1. Task Manager -->
        <section class="card">
            <h2>1. Task Manager (LocalStorage & OOP)</h2>
            <form id="task-form">
                <input type="text" id="task-input" placeholder="Enter new task..." required />
                <button type="submit">Add Task</button>
            </form>
            <ul id="task-list"></ul>
        </section>
        <!-- 2. Form Submission Module -->
        <section class="card">
            <h2>2. User Registration Form</h2>
            <form id="user-form">
                <input type="text" id="user-name" placeholder="Full Name" required />
                <input type="email" id="user-email" placeholder="Email Address" required />
                <select id="user-role" required>
                    <option value="" disabled selected>Select Role...</option>
                    <option value="Developer">Developer</option>
                    <option value="Designer">Designer</option>
                    <option value="Student">Student</option>
                </select>
                <button type="submit">Submit Details</button>
            </form>
            <div id="submissions-container" style="margin-top: 15px; max-height: 180px; overflow-y: auto;">
                <!-- Dynamic submissions render here -->
            </div>
        </section>
        <!-- 3. Sports & Interests Selection Module -->
        <section class="card">
            <h2>3. Sports & Interests Selector</h2>
            <form id="interests-form">
                <select id="category-select" required>
                    <option value="" disabled selected>Primary Interest Category...</option>
                    <option value="Outdoor Sports">Outdoor Sports</option>
                    <option value="Indoor Games">Indoor Games</option>
                    <option value="Fitness & Health">Fitness & Health</option>
                </select>
                <p style="font-size: 0.85rem; margin-top: 10px; color: #94a3b8;">Select your favorite sports:</p>
                <div class="checkbox-group">
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Football"> Football</label>
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Basketball"> Basketball</label>
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Tennis"> Tennis</label>
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Cricket"> Cricket</label>
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Swimming"> Swimming</label>
                    <label class="checkbox-label"><input type="checkbox" name="sport" value="Badminton"> Badminton</label>
                </div>
                <button type="submit">Save Preferences</button>
            </form>
            <div id="selected-tags" class="tags-container">
                <!-- Selected sport tags render here -->
            </div>
        </section>
    </main>
    <script>
        // =====
        // Utility: XSS Protection
        // =====
        function escapeHTML(str) {
            return str.replace(/[&<>"]/g,
                tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'"': '&apos;' }[tag] || tag)
            );
        }
    </script>

```

```

    );
}
// =====
// 1. OOP & LOCAL STORAGE (Task Manager)
// =====
class TaskManager {
  constructor(storageKey) {
    this.storageKey = storageKey;
    this.tasks = JSON.parse(localStorage.getItem(this.storageKey)) || [];
  }
  addTask(title) {
    const task = { id: Date.now(), title };
    this.tasks.push(task);
    this.save();
    return task;
  }
  deleteTask(id) {
    this.tasks = this.tasks.filter(task => task.id !== id);
    this.save();
  }
  save() {
    localStorage.setItem(this.storageKey, JSON.stringify(this.tasks));
  }
}
const appTasks = new TaskManager('my_app_tasks');
const taskForm = document.getElementById('task-form');
const taskInput = document.getElementById('task-input');
const taskList = document.getElementById('task-list');
function renderTasks() {
  taskList.innerHTML = '';
  appTasks.tasks.forEach(task => appendTaskDOM(task));
}
function appendTaskDOM(task) {
  const li = document.createElement('li');
  li.dataset.id = task.id;
  li.innerHTML = `
    <span>${escapeHTML(task.title)}</span>
    <button class="delete-btn">Delete</button>
  `;
  taskList.appendChild(li);
}
taskForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const title = taskInput.value.trim();
  if (title) {
    const newTask = appTasks.addTask(title);
    appendTaskDOM(newTask);
    taskInput.value = '';
  }
});
taskList.addEventListener('click', (e) => {
  if (e.target.classList.contains('delete-btn')) {
    const li = e.target.parentElement;
    const id = Number(li.dataset.id);
    appTasks.deleteTask(id);
    li.remove();
  }
});
// =====
// 2. USER REGISTRATION FORM
// =====
const userForm = document.getElementById('user-form');
const submissionsContainer = document.getElementById('submissions-container');
userForm.addEventListener('submit', (e) => {
  e.preventDefault();
  const name = document.getElementById('user-name').value.trim();
  const email = document.getElementById('user-email').value.trim();
  const role = document.getElementById('user-role').value;
  const submission = {
    name,
    email,
    role,
    time: new Date().toLocaleTimeString()
  };
  renderSubmissionDetails(submission);
  userForm.reset();
});
function renderSubmissionDetails(data) {
  const card = document.createElement('div');
  card.className = 'submission-card';
  card.innerHTML = `
    <div><strong>Name:</strong> ${escapeHTML(data.name)}</div>
    <div><strong>Email:</strong> ${escapeHTML(data.email)}</div>
    <div><strong>Role:</strong> ${escapeHTML(data.role)}</div>
    <span class="timestamp">Submitted at ${data.time}</span>
  `;

```

```

    `;
    submissionsContainer.prepend(card);
  }
  // =====
  // 3. SPORTS & INTERESTS SELECTOR
  // =====
  const interestsForm = document.getElementById('interests-form');
  const selectedTagsContainer = document.getElementById('selected-tags');
  interestsForm.addEventListener('submit', (e) => {
    e.preventDefault();
    const category = document.getElementById('category-select').value;
    const checkedBoxes = document.querySelectorAll('input[name="sport"]:checked');
    const selectedSports = Array.from(checkedBoxes).map(cb => cb.value);
    if (selectedSports.length === 0) {
      alert('Please select at least one sport!');
      return;
    }
    renderInterestTags(category, selectedSports);
  });
  function renderInterestTags(category, sports) {
    selectedTagsContainer.innerHTML = ''; // Clear previous tags
    // Render Category Badge
    const catBadge = document.createElement('div');
    catBadge.style.width = '100%';
    catBadge.style.fontSize = '0.85rem';
    catBadge.style.color = '#94a3b8';
    catBadge.innerHTML = `Category: <strong style="color:var(--accent);">${escapeHTML(category)}</strong>`;
    selectedTagsContainer.appendChild(catBadge);
    // Render Pill for each selected sport
    sports.forEach(sport => {
      const pill = document.createElement('div');
      pill.className = 'tag-pill';
      pill.innerHTML = `
        ${escapeHTML(sport)}
        <button type="button" class="remove-tag">&times;</button>
      `;
      selectedTagsContainer.appendChild(pill);
    });
  }
  // Remove individual pill on click
  selectedTagsContainer.addEventListener('click', (e) => {
    if (e.target.classList.contains('remove-tag')) {
      e.target.parentElement.remove();
    }
  });
  // Initialize state on load
  renderTasks();
</script>
</body>
</html>

```

7. CODE EXPLANATION

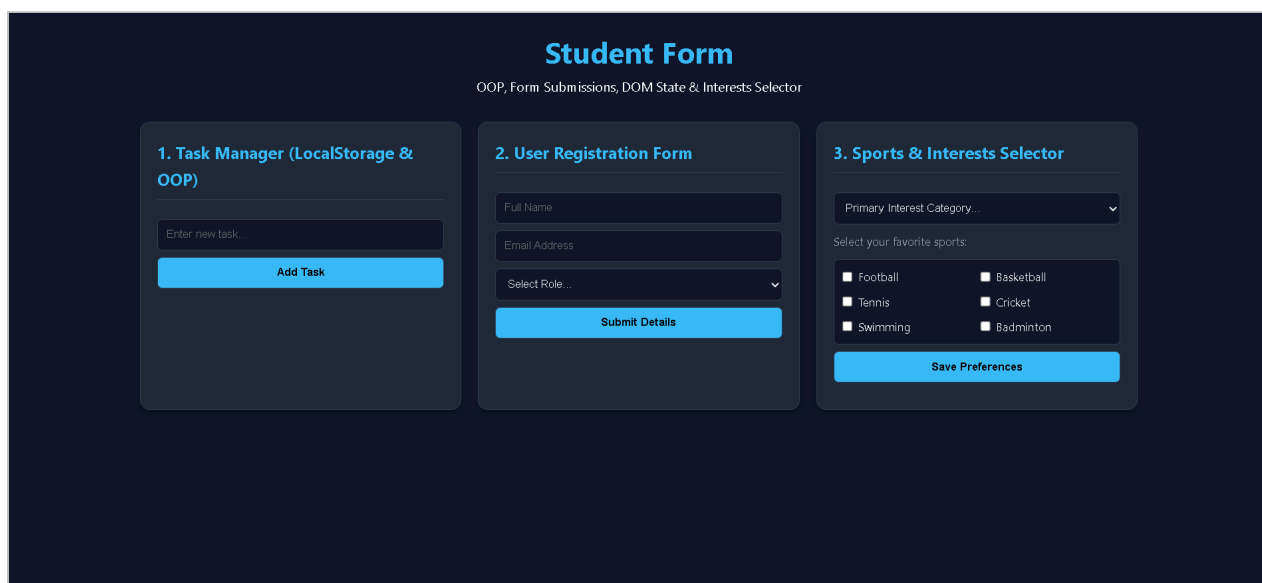
Code / Element	Definition
class TaskManager	Defines a class (OOP) that manages the list of tasks and their storage.
constructor(storageKey)	Initialises the object and loads saved tasks from LocalStorage (or an empty array).
addTask(title) / deleteTask(id)	Class methods that add a task object to the array and remove a task by its id.
save()	Stores the tasks array in LocalStorage using JSON.stringify().
localStorage.getItem() / setItem()	Reads and writes data in the browser so that it remains after a page reload.
JSON.parse() / JSON.stringify()	Converts stored text to a JavaScript array and the array back to text.
Date.now()	Generates a unique numeric id for each task.
document.getElementById()	Selects HTML elements using their unique IDs.
document.querySelectorAll()	Selects all elements matching a CSS selector, such as the checked sport checkboxes.
addEventListener('submit')	Runs a function when a form is submitted.
e.preventDefault()	Stops the default form submission so the page does not reload.
.value / .trim()	Gets the entered value of an input field and removes extra spaces.
document.createElement()	Creates a new HTML element (li, div) dynamically.
innerHTML	Sets or clears the HTML content of an element.
appendChild() / prepend()	Adds a new element at the end or at the beginning of a parent element.
li.dataset.id	Stores the task id in a data attribute of the list item.
e.target.classList.contains()	Checks which button was clicked (event delegation on the parent element).
li.remove()	Removes an element from the DOM.
Array.filter()	Creates a new array without the deleted task.
Array.from() / map()	Converts the checked checkboxes into an array of sport names.
forEach()	Iterates through every task or selected sport to display it.
userForm.reset()	Clears all fields of the registration form after submission.
new Date().toLocaleTimeString()	Gets the current time to show as the submission time.
alert()	Displays a message if no sport is selected.
escapeHTML()	Replaces special characters (&, <, >, quotes) to prevent XSS attacks.
CSS variables (:root)	Stores theme colours once and reuses them using var().
display: grid / flex	Creates the responsive card layout, checkbox grid, and tag list.

8. TEST CASES AND EXECUTION DATA

The Student Form dashboard was executed in a web browser and tested for adding, deleting, and persisting tasks, form submission, sports selection, removing tags, and input escaping.

Test Case	TC-01
Test / Input	Open the Student Form dashboard webpage in the browser
Expected Result	The dashboard should be displayed with the Task Manager, User Registration Form, and Sports & Interests Selector cards, each with its input fields and button.
Actual Result	The Student Form dashboard was displayed correctly with all three modules and their components.
Status	Pass

TC-01 Output:



The screenshot displays the 'Student Form' dashboard with the subtitle 'OOP, Form Submissions, DOM State & Interests Selector'. It features three main modules:

- 1. Task Manager (LocalStorage & OOP):** Includes an input field 'Enter new task...' and an 'Add Task' button.
- 2. User Registration Form:** Includes input fields for 'Full Name', 'Email Address', and a 'Select Role...' dropdown, followed by a 'Submit Details' button.
- 3. Sports & Interests Selector:** Includes a 'Primary Interest Category...' dropdown, a section to 'Select your favorite sports' with checkboxes for Football, Basketball, Tennis, Cricket, Swimming, and Badminton, and a 'Save Preferences' button.

Test Case	TC-02
Test / Input	Enter the tasks <i>Complete FSWD Experiment 5</i> , <i>Prepare for viva</i> and <i>Submit lab record</i> in the Task Manager and click Add Task
Expected Result	Each task should be added to the task list dynamically without reloading the page, and the input box should be cleared.
Actual Result	All three tasks were added to the list instantly with a Delete button beside each, and the input field was cleared.
Status	Pass

TC-02 Output:

Student Form

OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Add Task

Complete FSWD Experiment 5Delete

Prepare for vivaDelete

Submit lab recordDelete

2. User Registration Form

Select Role...

Submit Details

3. Sports & Interests Selector

Primary Interest Category...

Select your favorite sports:

☐ Football☐ Basketball

☐ Tennis☐ Cricket

☐ Swimming☐ Badminton

Save Preferences

Test Case	TC-03
Test / Input	Click the Delete button beside the task <i>Prepare for viva</i>
Expected Result	The selected task should be removed from the list and from the LocalStorage data, while the other tasks remain.
Actual Result	The task was removed from the list immediately and only the remaining two tasks were displayed.
Status	Pass

TC-03 Output:

Student Form

OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Add Task

Complete FSWD Experiment 5Delete

Submit lab recordDelete

2. User Registration Form

Select Role...

Submit Details

3. Sports & Interests Selector

Primary Interest Category...

Select your favorite sports:

☐ Football☐ Basketball

☐ Tennis☐ Cricket

☐ Swimming☐ Badminton

Save Preferences

Test Case	TC-04
Test / Input	Reload (refresh) the webpage after adding and deleting tasks
Expected Result	The remaining tasks should be loaded again from LocalStorage and displayed in the task list.
Actual Result	The tasks were retained after reloading the page, showing that LocalStorage persisted the data.
Status	Pass

TC-04 Output:

Student Form

OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Enter new task...

Add Task

Complete FSWD Experiment 5Delete

Submit lab recordDelete

2. User Registration Form

Full Name

Email Address

Select Role...

Submit Details

3. Sports & Interests Selector

Primary Interest Category...

Select your favorite sports:

☐ Football

☐ Basketball

☐ Tennis

☐ Cricket

☐ Swimming

☐ Badminton

Save Preferences

Test Case	TC-05
Test / Input	Enter Full Name <i>K. Suhas Varma</i> , Email <i>suhas.varma@example.com</i> , select Role <i>Student</i> and click Submit Details
Expected Result	A submission card showing the name, email, role, and submission time should appear below the form, and the form should be reset.
Actual Result	The submission card was displayed with the entered Name, Email, Role, and time of submission, and the form fields were cleared.
Status	Pass

TC-05 Output:

Student Form

OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Enter new task...

Add Task

Complete FSWD Experiment 5Delete

Submit lab recordDelete

2. User Registration Form

Full Name

Email Address

Select Role...

Submit Details

Name: K. Suhas Varma

Email: suhas.varma@example.com

Role: Student

Submitted at 2:54:52 PM

3. Sports & Interests Selector

Primary Interest Category...

Select your favorite sports:

☐ Football

☐ Basketball

☐ Tennis

☐ Cricket

☐ Swimming

☐ Badminton

Save Preferences

Test Case	TC-06
Test / Input	Select the category <i>Outdoor Sports</i> , tick <i>Football</i> , <i>Cricket</i> and <i>Badminton</i> , and click Save Preferences
Expected Result	The selected category and a tag (pill) for each selected sport should be displayed below the form.
Actual Result	The category <i>Outdoor Sports</i> and the tags Football, Cricket and Badminton were displayed correctly.
Status	Pass

TC-06 Output:

Student Form
OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Enter new task...

Add Task

Complete FSWD Experiment 5 Delete

Submit lab record Delete

2. User Registration Form

Full Name

Email Address

Select Role...

Submit Details

Name: K. Suhas Varma
Email: suhas.varma@example.com
Role: Student
Submitted at 2:54:52 PM

3. Sports & Interests Selector

Outdoor Sports

Select your favorite sports:

☒ Football ☐ Basketball
☐ Tennis ☒ Cricket
☐ Swimming ☒ Badminton

Save Preferences

Category: Outdoor Sports

Football × Cricket × Badminton ×

Test Case	TC-07
Test / Input	Click the × button on the <i>Football</i> tag
Expected Result	Only the clicked sport tag should be removed from the selected tags, while the other tags remain.
Actual Result	The Football tag was removed and the Cricket and Badminton tags remained displayed.
Status	Pass

TC-07 Output:

Student Form
OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Enter new task...

Add Task

Complete FSWD Experiment 5 Delete

Submit lab record Delete

2. User Registration Form

Full Name

Email Address

Select Role...

Submit Details

Name: K. Suhas Varma
Email: suhas.varma@example.com
Role: Student
Submitted at 2:54:52 PM

3. Sports & Interests Selector

Outdoor Sports

Select your favorite sports:

☒ Football ☐ Basketball
☐ Tennis ☒ Cricket
☐ Swimming ☒ Badminton

Save Preferences

Category: Outdoor Sports

Cricket × Badminton ×

Test Case	TC-08
Test / Input	Add the task text <code>bold </code> in the Task Manager
Expected Result	The text should be shown exactly as entered (escaped) and no HTML or script should be executed (XSS protection using <code>escapeHTML()</code>).
Actual Result	The text was displayed as plain text; no bold formatting, image, or alert was executed.
Status	Pass

TC-08 Output:

Student Form

OOP, Form Submissions, DOM State & Interests Selector

1. Task Manager (LocalStorage & OOP)

Add Task

Complete FSWD Experiment 5

Delete

Submit lab record

Delete

`<b>bold</b> <img src=x onerror=alert(1)>`

Delete

2. User Registration Form

Select Role...

Submit Details

Name: K. Suhas Varma

Email: suhas.varma@example.com

Role: Student

Submitted at 2:54:52 PM

3. Sports & Interests Selector

Outdoor Sports

Select your favorite sports:

☒ Football

☐ Basketball

☐ Tennis

☒ Cricket

☐ Swimming

☒ Badminton

Save Preferences

Category: Outdoor Sports

Cricket

Badminton

9. OUTPUT

Output: Student Form Dashboard Webpage

The following screenshot shows the complete execution of the **Student Form dashboard**. It displays the Task Manager, User Registration Form, and Sports & Interests Selector modules.

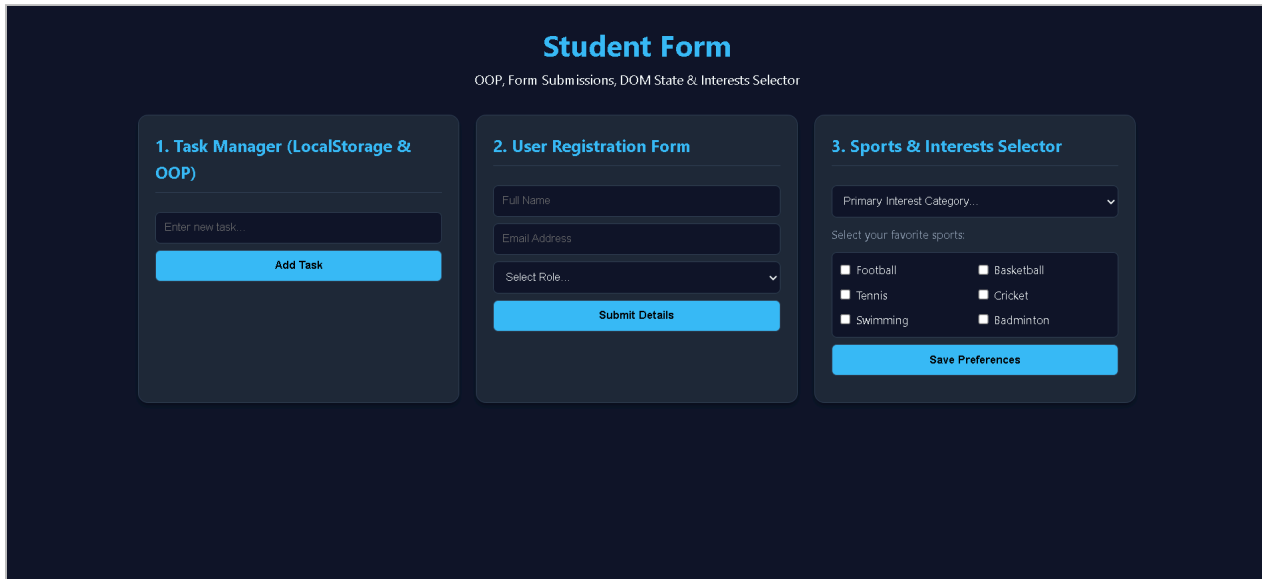
The screenshot displays a web application titled "Student Form" with a subtitle "OOP, Form Submissions, DOM State & Interests Selector". The interface is divided into three main sections: 1. Task Manager (LocalStorage & OOP), which includes a text input for "Enter new task..." and an "Add Task" button; 2. User Registration Form, which includes input fields for "Full Name", "Email Address", a "Select Role..." dropdown menu, and a "Submit Details" button; 3. Sports & Interests Selector, which includes a "Primary Interest Category..." dropdown, a section for "Select your favorite sports" with checkboxes for Football, Tennis, Swimming, Basketball, Cricket, and Badminton, and a "Save Preferences" button. The entire application is styled with a dark blue background and light blue accents.

Figure 1: Output of Student Form Dashboard Using DOM and JavaScript

10. RESULT

Thus, a dynamic **Student Form dashboard webpage** was successfully developed using **HTML, CSS, JavaScript, and DOM manipulation**. The application successfully allows users to add and delete tasks with LocalStorage persistence using an OOP class, submit registration details and display them dynamically, and select sports and interests as removable tags. Input validation, event handling, DOM element creation, XSS protection, and array operations were implemented successfully. The webpage was executed and verified using multiple test cases, and the expected results were obtained.